

Lab Ten – Point Estimation

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with “**Solution:**”.
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use “enter” to make your code readable, so it doesn’t run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you’re submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don’t forget that you can copy-and-paste code when you’re doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good **ggplot** practice AND will help make sure you don’t make a mistake when computing the probability.

1 Lab 8 Notes

1.1 Altham’s Multiplicative Binomial Distribution

The PMF for Altham’s Multiplicative Binomial Distribution is

$$f_X(x) = \frac{1}{c} \binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)} I(x \in \{0, 1, \dots, n\})$$

where

$$c = \sum_{i=0}^n \binom{n}{i} p^i (1-p)^{n-i} \theta^{i(n-i)}.$$

Just so we start on the same page, I have included my R function for computing probability using this distribution.

```
1 daltbinom <- function(x, size, prob, theta){
2   normalizing.constant <- 0
3   for(j in 0:size){
4     normalizing.constant <- normalizing.constant +
5       choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
6   }
7   # Probability Mass at x=x
8   (1/normalizing.constant) *
9     choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
10     (x>=0)*(x<=size)
11 }
```

2 Altham’s Multiplicative Beta Binomial Distribution

The PMF for Altham’s Multiplicative Beta Binomial Distribution is

$$f_X(x) = \left[\int_0^1 \left(\frac{1}{c} \binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)} \right) \left(\frac{\Gamma(\alpha + \beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1} (1-p)^{\beta-1} \right) dp \right] I(x \in \{0, 1, \dots, n\})$$

Just so we start on the same page, I have included my R function for computing probability using this distribution.

```

1 daltbbinom.single <- function(x, size, alpha, beta, theta){
2   integrand <- function(p) {
3     sapply(p, # integrate will pass in a vector, so we use sapply to apply the following
4       function(p_val){
5         # f(x|p) * f(p|alpha, beta)
6         daltbinom(x, size, p_val, theta) * dbeta(p_val, alpha, beta)
7       }
8   )
9 }
10 # Probability Mass at x=x
11 # integrate(integrand, lower = 0, upper = 1)$value * (x>=0)*(x<=size)
12 # I shrink the bounds below to avoid log(0)=-infinty
13 # I use try() to catch when there is an error and avoid crashing
14 res <- try(integrate(integrand, lower = 1e-7, upper = 1 - 1e-7)$value *
15   (x>=0)*(x<=size),
16   silent = TRUE)
17 # If there is an error, return a near-zero probability
18 if(inherits(res, "try-error")) return(0.1^6)
19
20 res
21 }
22 # Write a function that works on a vector
23 daltbbinom <- function(x, size, alpha, beta, theta){
24   sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
25 }

```

3 Question 1

In Lab 8, we use Altham's Multiplicative Binomial Distribution. Your job was to interpret the impact of θ in that equation. In this lab, we will model the number of legs won in a best of eleven (first to six) match.

To help us think about the interpretation, something that was challenging before, let's try something a little more manageable.

3.1 Part a

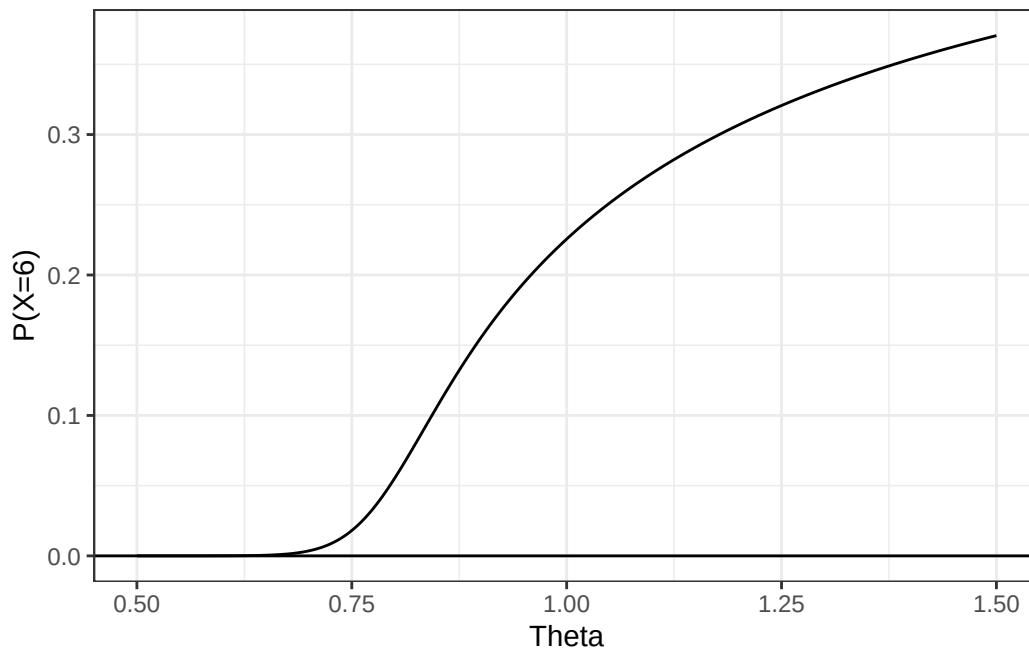
Suppose a player has a 50% chance of winning a specific leg, that is let $p = 0.50$. Let's look at the probability they win ($x = 6$) legs in the match. Compute this probability over a sequence from $\theta = 0.5$ to $\theta = 1.5$ and plot it using a line where the x -axis is θ and the y -axis is $P(X = 6)$.

Solution

```

1 ggdat <- tibble(theta = seq(0.5, 1.5, 0.001)) |>
2   mutate(Probability = daltbinom(6, size = 11, prob= 0.5, theta = theta))
3
4 plot <- ggplot(ggdat) +
5   geom_line(aes(x = theta, y= Probability)) +
6   geom_hline(yintercept = 0) +
7   ylab("P(X=6)") +
8   xlab("Theta") +
9   theme_bw()
10
11 plot

```



3.2 Part b

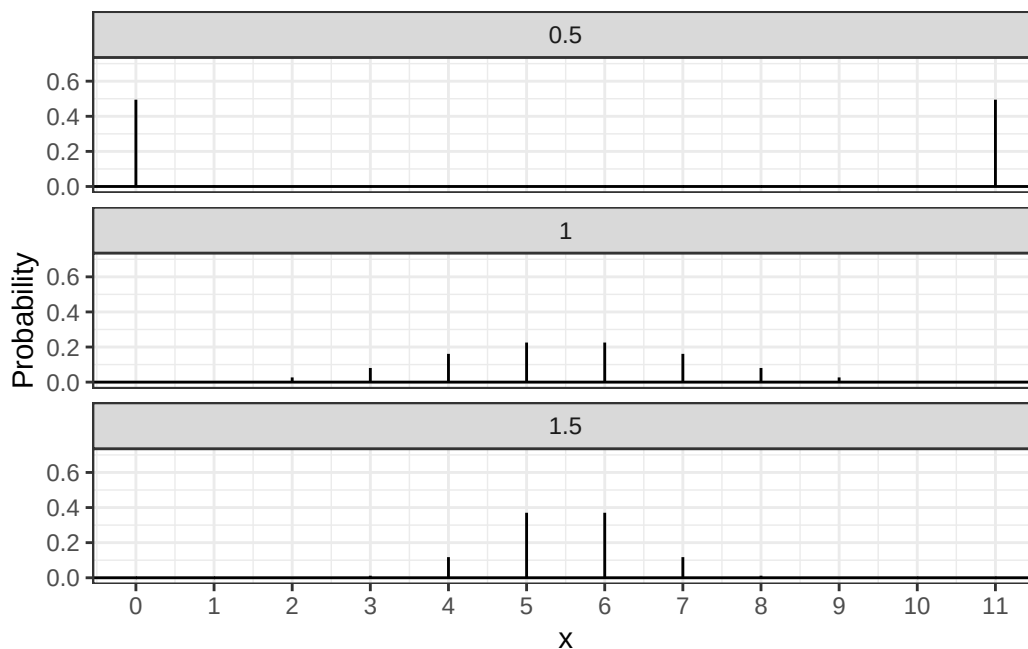
Now, plot Altham's Multiplicative Binomial Distribution under the same parameters with $\theta = 0.50$, $\theta = 1$, and $\theta = 1.5$. Use `ncol=1` in `facet_wrap()` to plot the PMFs in a single column for comparing.

Solution

```

1 ggdat <- tibble(x = rep(seq(0, 11, 1), 3), theta = c(rep(0.5, 12), rep(1, 12), rep(1.5, 12))) |>
2   mutate(PMF= daltbinom(x, 11, 0.5, theta))
3
4 PMF.plot <- ggplot() +
5   geom_linerange(data = ggdat,
6                 aes(x = x, ymin = 0, ymax = PMF)) +
7   geom_hline(yintercept = 0) +
8   scale_x_continuous("x", breaks=0:11) +
9   scale_y_continuous(limits = c(0, 0.7)) +
10  ylab("Probability") +
11  facet_wrap(~theta, ncol = 1) +
12  theme_bw()
13
14 PMF.plot

```



3.3 Part c

What is your best assessment of what θ is doing in this setting. Consider when a 6-0 blowout is most likely, and when a 5-6 nail-biter is most likely. Note here we should treat the match as a fixed $n = 11$ as we did for $n = 3$ in Lab 8.

Note: In lab 8, we saw that small θ meant wins came in bunches (clumped together), so there were more 2-0 wins in a best of three series.

Solution When θ is greater, x , or the number of matches win, are likely to have higher probability at the center, which means winning 5 or 6 matches are more likely. This means when θ is lower, 6-0 is more likely and when θ is higher, 6-5 are more likely.

4 Question 2

4.1 Part a

Altham's Multiplicative Binomial Distribution does not typically simplify to the clean $E(X) = np$ seen in the Binomial distribution, unless $\theta = 1$. Write out the expression that would need to be solved to find $E(X^k)$, and then write a function that computes it given the proposed parameter values p (`prob`) and θ (`theta`).

Solution

$$E(X^K) = \sum_{i=0}^{11} i^k f_x(i)$$

```

1 momdaltbinom <- function(p, theta, k){
2   x <- 0:11
3   sum(x^k*daltbinom(x,11,p,theta))
4 }

```

4.2 Part b

Describe how you can use your answer to part a to find Method of Moments estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution

We can calculate the $\bar{x}$ and set it equal to $\sum_{i=0}^{11} x_i f_x(x_i)$; calculate $\overline{x^2}$ and set it equal to $\sum_{i=0}^{11} x_i^2 f_x(x_i)$ and solve for p and θ .

4.3 Part c

Write a function that computes the log-likelihood for Altham's Multiplicative Binomial Distribution, given data ($\mathbf{x}$) and proposed parameter values p (`prob`) and θ (`theta`).

Solution

```
1 lldaltbinom <- function(par, data, MLE = F){  
2   p <- ifelse(MLE, 1/(1+exp(-par[1])), par[1])  
3   theta <- ifelse(MLE, exp(par[2]), par[2])  
4   ll <- sum(log(daltbinom(data,11,p,theta)))  
5   ifelse(MLE, -ll, ll)  
6 }
```

4.4 Part d

Describe how you can use your answer to part c to find Maximum Likelihood estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution

Use `optim()` to find the parameter that allows the log likelihood function to have the greatest value, and set `MLE` in the function equal to `True`.

5 Lab 10

Peter “Snakebite” Wright is a two-time Professional Darts Corporation World Champion (2020, 2022) and a former World No. 1. He is one of the sport's most decorated players, winning nearly 50 PDC titles including the World Matchplay, UK Open, and multiple European Championships.

In 2024, he made The Premier League – a league involving the eight best players that runs every Thursday night from February to May. In this season, he won two of eighteen matches, securing only four points. The spread in legs won was -43, meaning he lost 43 more legs than he won. This performance was rather quite bad. He was soundly trounced in almost all his match ups.

The 2024 standings are below:

Rank	Player
1	Luke Littler
2	Luke Humphries
3	Michael van Gerwen
4	Michael Smith
5	Nathan Aspinall
6	Rob Cross
7	Gerwyn Price
8	Peter Wright

5.1 Summarize the 2024 Data

In `pd2024.csv`, I have provided the data for the 2024 Premier League season. There are a few data cleaning steps to consider.

- Remove any matches with no `Score` (e.g., a bye or walkover)
- Nights 1 through 16 are best of 11 (first to 6), but the playoffs are extended. For consistency, keep nights 1 through 16 only.
- Make two columns `WinnerLegs` and `LoserLegs` that keep track of the number of legs each player has won

Summarize the data. What do the data tell us about the breakdown of the results?

Solution

```

1 dat.pdc <- read_csv("pdc2024.csv")
2
3 dat.pdc.clean <- dat.pdc |>
4   filter(Score != "w/o") |>
5   filter(Night != "Finals") |>
6   mutate(WinnerLegs = as.numeric(str_split(Score, "-", simplify = TRUE)[,1]),
7           LoserLegs = as.numeric(str_split(Score, "-", simplify = TRUE)[,2]))
8
9 Matchwins <- function(name){
10   win <- 0
11   for(i in 1:nrow(dat.pdc.clean)){
12     if(dat.pdc.clean$Winner[i]==name){
13       win <- win + 1
14     }
15   }
16   win
17 }
18
19 total <- function(name){
20   total <- 0
21   for(i in 1:nrow(dat.pdc.clean)){
22     if(dat.pdc.clean$Winner[i]==name | dat.pdc.clean$Loser[i]==name){
23       total <- total + 1
24     }
25   }
26   total
27 }
28
29 legwins <- function(name){
30   legwin <- 0
31   for(i in 1:nrow(dat.pdc.clean)){
32     if(dat.pdc.clean$Winner[i]==name){
33       legwin <- legwin + 6
34     }
35     else if(dat.pdc.clean$Loser[i]==name){
36       legwin <- legwin + dat.pdc.clean$LoserLegs[i]
37     }
38   }
39   legwin
40 }
41
42 legtotal <- function(name){
43   total <- 0
44   for(i in 1:nrow(dat.pdc.clean)){
45     if(dat.pdc.clean$Winner[i]==name | dat.pdc.clean$Loser[i]==name){
46       total <- total + dat.pdc.clean$LoserLegs[i] + dat.pdc.clean$WinnerLegs[i]
47     }
48   }
49   total
50 }
51
52 player.turns <- table(c(dat.pdc.clean$Winner, dat.pdc.clean$Loser))
53 match.summary <- tibble(Name = names(player.turns),
54                           Match.Win = apply(names(player.turns), Matchwins),
55                           Match.Total = apply(names(player.turns), total)) |>
56   mutate(Proportion = Match.Win / Match.Total)
57
58
59

```

```

50 leg.summary <- tibble(Name = names(player.turns),
51                       Leg.Win = sapply(names(player.turns), legwins),
52                       Leg.Total = sapply(names(player.turns), legtotal)) |>
53                       mutate(Proportion = Leg.Win / Leg.Total)
54
55 match.plot <- ggplot(data = match.summary) +
56   geom_bar(aes(x = Name, y = Proportion), stat = "identity") +
57   geom_hline(yintercept = 0) +
58   theme_bw() +
59   labs(x = "Player Name", y = "Winning Matches Proportion") +
60   theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 10))
61
62 leg.plot <- ggplot(data = leg.summary) +
63   geom_bar(aes(x = Name, y = Proportion), stat = "identity") +
64   geom_hline(yintercept = 0) +
65   theme_bw() +
66   labs(x = "Player Name", y = "Winning Legs Proportion") +
67   theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 10))

```

Table 2: Matches won by each player

Name	Match.Win	Match.Total	Proportion
Gerwyn Price	7	22	0.32
Luke Humphries	20	32	0.62
Luke Littler	24	36	0.67
Michael Smith	17	31	0.55
Michael van Gerwen	16	28	0.57
Nathan Aspinall	15	29	0.52
Peter Wright	2	18	0.11
Rob Cross	10	26	0.38

```
1 match.plot
```

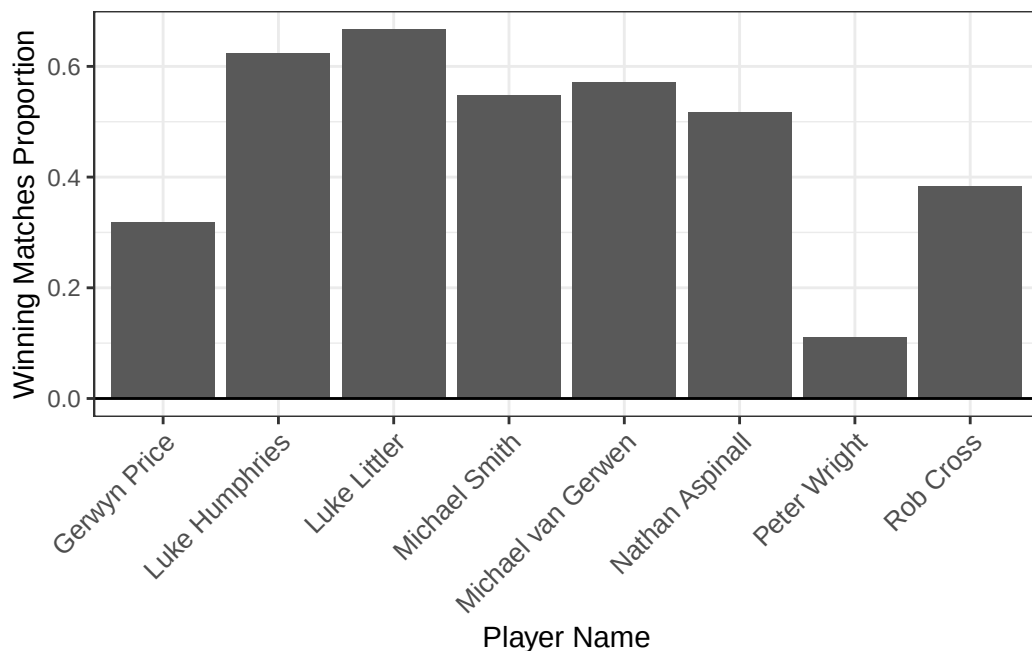
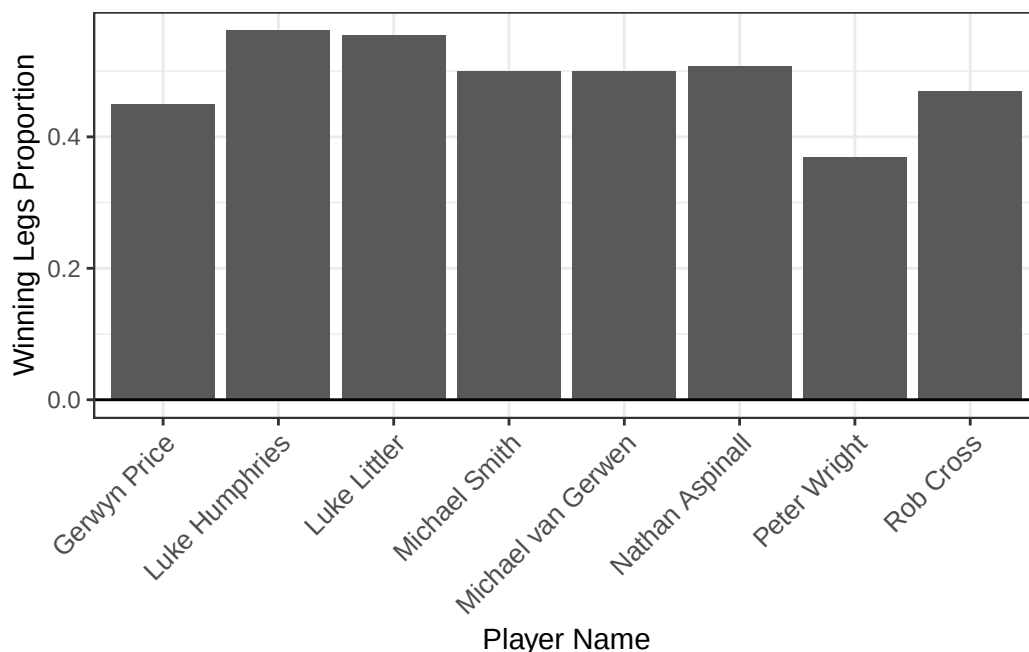


Table 3: Legs won by each player

Name	Leg.Win	Leg.Total	Proportion
Gerwyn Price	95	211	0.45
Luke Humphries	169	301	0.56
Luke Littler	193	348	0.55
Michael Smith	150	300	0.50
Michael van Gerwen	135	270	0.50
Nathan Aspinall	143	282	0.51
Peter Wright	61	165	0.37
Rob Cross	115	245	0.47

1 `leg.plot`



I use the data to calculate the total number of matches played by each player and the number of matches they won. I also compute their match winning proportions. The results show that Peter Wright has the lowest winning proportion, while Luke Littler has the highest.

Additionally, I calculate the total number of legs played by each player and the number of legs they won. There is less variation in leg winning proportions compared to match winning proportions. Although Peter Wright still has the lowest proportion, his leg winning proportion is a lot higher than his match winning proportion, but it is still lower than other players.

5.2 Altham's Multiplicative Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximum likelihood estimates for p and θ .

Note: The “for each” here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLE for the player using this data. Ensure to report p and θ for each player in a nicely formatted table.

Note: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space. For example, here, you might consider `p <- 1/(1+exp(-par[1]))` and `theta <- exp(par[2])`.

- Make comments about the MLEs in the context of the standings.

```

1 player.turns <- table(c(dat.pdc.clean$Winner, dat.pdc.clean$Loser))
2
3 # This function returns a data frame that contains two column.
4 # First column is number of legs the player wins in each match,
5 # and the second column is the total legs that the player played for each match.
6 Win.each <- function(name){
7   total.turn <- as.numeric(player.turns[name])
8   win.each <- tibble(Win = rep(NA, total.turn),
9                       Total = rep(NA, total.turn))
10  n <- 1
11  for(i in 1:nrow(dat.pdc.clean)){
12    if(dat.pdc.clean$Winner[i]==name){ # If the player is the winner
13      win.each$Win[n] <- 6
14      win.each$Total[n] <- dat.pdc.clean$LoserLegs[i] + dat.pdc.clean$WinnerLegs[i]
15      n <- n + 1
16    }
17    else if(dat.pdc.clean$Loser[i]==name){ # If the player is the loser
18      win.each$Win[n] <- dat.pdc.clean$LoserLegs[i]
19      win.each$Total[n] <- dat.pdc.clean$LoserLegs[i] + dat.pdc.clean$WinnerLegs[i]
20      n <- n + 1
21    }
22  }
23  win.each
24 }
25
26 # This function calculates the probability of the player
27 # winning x time in n legs with given probability and theta.
28 # If the player win the match, it ensures the last leg is a win,
29 # If the player lose the match, it ensures the last leg is a lose.
30 # Since player did not play 11 legs, size is also a vector that contains different total legs each player p
31
32 Prob.daltbinom <- function(x, size, prob, theta){
33   sapply(1:length(x), function(i){ # Ensure it works on a vector
34     xi <- x[i]
35     si <- size[i]
36     if(xi == 6){ # If the player win the match
37       # Calculate the probability and ensuring the last leg is win
38       (choose(si-1,5)/choose(si, xi)) * daltbinom(xi, si, prob, theta)
39     } else { # If the player lose the match
40       # Calculate the probability and ensuring the last leg is lose
41       (choose(si-1, xi)/choose(si, xi)) * daltbinom(xi, si, prob, theta)
42     }
43   })
44 }
45
46 # This function calculates the log likelihood of different parameters.
47 # In order to make it easier for optimization, it returns the negative log likelihood.
48 llProb.daltbinom <- function(par, data){
49   p <- 1/(1+exp(-par[1])) # Ensures probability is between 0 and 1
50   theta <- exp(par[2]) # Ensures theta is greater than 0
51   x <- data$Win
52   size <- data$Total
53   ll <- sum(log(Prob.daltbinom(x,size,p,theta)))
54   -ll
55 }

```

```

56 # Create a summary table for the result
57 MLE.summary <- tibble(Name = names(player.turns),
58                       P = NA,
59                       Theta = NA)
60
61 # Repeat the process for each player
62 for(i in 1:nrow(MLE.summary)){
63   name <- MLE.summary$Name[i]
64   win.each <- Win.each(name)
65   mle <- optim(par = c(0,0), fn = llProb.daltbinom,
66              data = win.each)
67   MLE.summary$P[i] <- 1/(1+exp(-mle$par[1]))
68   MLE.summary$Theta[i] <- exp(mle$par[2])
69 }
70

```

Table 4: P and Theta for each player

Name	P	Theta
Gerwyn Price	0.43	1.12
Luke Humphries	0.58	1.08
Luke Littler	0.59	1.14
Michael Smith	0.50	1.11
Michael van Gerwen	0.50	1.11
Nathan Aspinall	0.51	1.14
Peter Wright	0.24	1.30
Rob Cross	0.46	1.04

The data represent the maximum likelihood of p and theta for each player. It means each player are most likely to have p probability of winning each match if their results follows Altham's Multiplicative Binomial Distribution.

5.3 Altham's Multiplicative Beta Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximum likelihood estimates for p and θ .

Note: The “for each” here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLEs for the player using this data. Ensure to report α , β , and θ for each player in a nicely formatted table.

Note 1: Due to the computational complexity of `integrate()`, you should compute the logged probability for 0:6 once and add them according to the data in each player's vector. **Note 2:** In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space.

- Plot the underlying Beta(α , β) distribution for each player.

Note: Add the mean and variance of the Beta to the table of MLEs. Recall

$$E(X) = \frac{\alpha}{\alpha + \beta}$$

$$sd(X) = \sqrt{\frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}}$$

- Make comments about the MLEs in the context of the standings.

Note: The players alternate who throws first in a leg has the mathematical “head start” to reach a finish before their opponent can take another turn. Players alternate who throws first with each leg.

Solution

```

1 # This function calculates the probability of the player
2 # wins x time with given alpha, beta, and theta.
3 Prob.daltbbinom <- function(x, size, alpha, beta, theta){
4   sapply(1:length(x), function(i){ # Ensure it works on a vector
5     xi <- x[i]
6     si <- size[i]
7     if(xi == 6){ # If the player win the match
8       # Calculate the probability and ensuring the last leg is win
9       (choose(si-1,5)/choose(si, xi)) * daltbbinom(xi, si, alpha, beta, theta)
10    } else { # If the player lose the match
11      # Calculate the probability and ensuring the last leg is lose
12      (choose(si-1, xi)/choose(si, xi)) * daltbbinom(xi, si, alpha, beta, theta)
13    }
14  })
15 }
16
17
18
19 # This function calculates the log likelihood of different parameters.
20 # In order to make it easier for optimization, it returns the negative log likelihood.
21 llProb.daltbbinom <- function(par, data){
22   alpha <- exp(par[1])
23   beta <- exp(par[2])
24   theta <- exp(par[3])
25   x <- data$Win
26   size <- data$Total
27   ll <- sum(log(Prob.daltbbinom(x,size,alpha, beta,theta)))
28   -ll
29 }
30
31 # Create a summary table for the result
32 MLE.summary <- tibble(Name = names(player.turns),
33                        Alpha = NA,
34                        Beta = NA,
35                        Theta = NA,
36                        Mean = NA,
37                        sd = NA)
38
39 # Repeat the process for each player
40 for(i in 1:nrow(MLE.summary)){
41   name <- MLE.summary$Name[i]
42   win.each <- Win.each(name)
43   mle <- optim(par = c(0,0,0), fn = llProb.daltbbinom,
44              data = win.each)
45   alpha <- exp(mle$par[1])
46   beta <- exp(mle$par[2])
47   MLE.summary$Alpha[i] <- alpha
48   MLE.summary$Beta[i] <- beta
49   MLE.summary$Theta[i] <- exp(mle$par[3])
50   MLE.summary$Mean[i] <- alpha / (alpha + beta)
51   MLE.summary$sd[i] <- ((alpha * beta)/((alpha + beta)^2 * (alpha + beta + 1))) ^ (1/2)
52 }

```

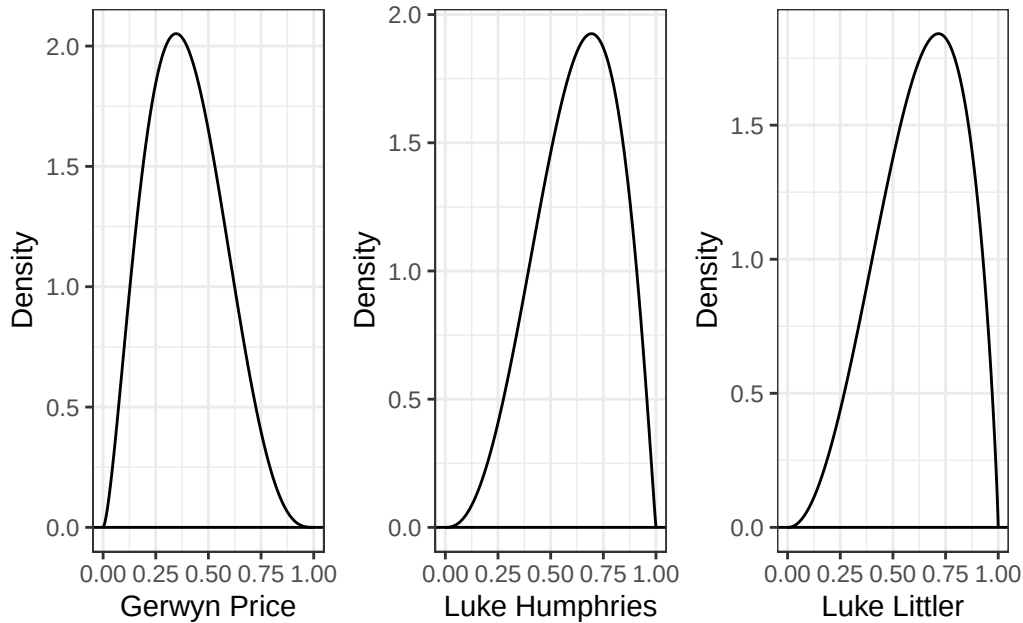
Table 5: Alpha, Beta and Theta for each player

Name	Alpha	Beta	Theta	Mean	sd
Gerwyn Price	2.58	3.98	1.38	0.39	0.18
Luke Humphries	3.48	2.10	1.36	0.62	0.19
Luke Littler	3.15	1.85	1.50	0.63	0.20
Michael Smith	2.67	2.70	1.41	0.50	0.20
Michael van Gerwen	3.46	3.51	1.34	0.50	0.18
Nathan Aspinall	3.92	3.66	1.37	0.52	0.17
Peter Wright	2.39	9.88	1.56	0.19	0.11
Rob Cross	1.86	2.34	1.36	0.44	0.22

```

1 Beta.plot <- function(i){
2   alpha <- MLE.summary$Alpha[i]
3   beta <- MLE.summary$Beta[i]
4
5   ggdat <- tibble(x = seq(0, 1, 0.001)) |>
6   mutate(PDF = dbeta(x, alpha, beta))
7
8   PDF.plot <- ggplot(ggdat) +
9     geom_line(aes(x = x, y= PDF)) +
10    geom_hline(yintercept = 0) +
11    xlab(MLE.summary$Name[i]) +
12    ylab("Density") +
13    theme_bw()
14   PDF.plot
15 }
16
17 Beta.plot(1) | Beta.plot(2) | Beta.plot(3)

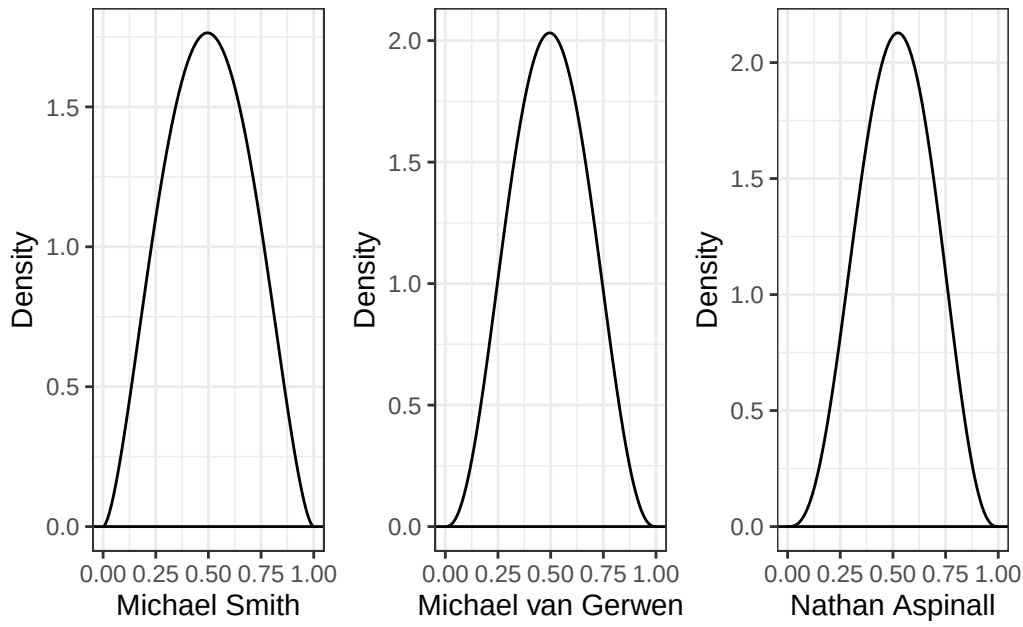
```



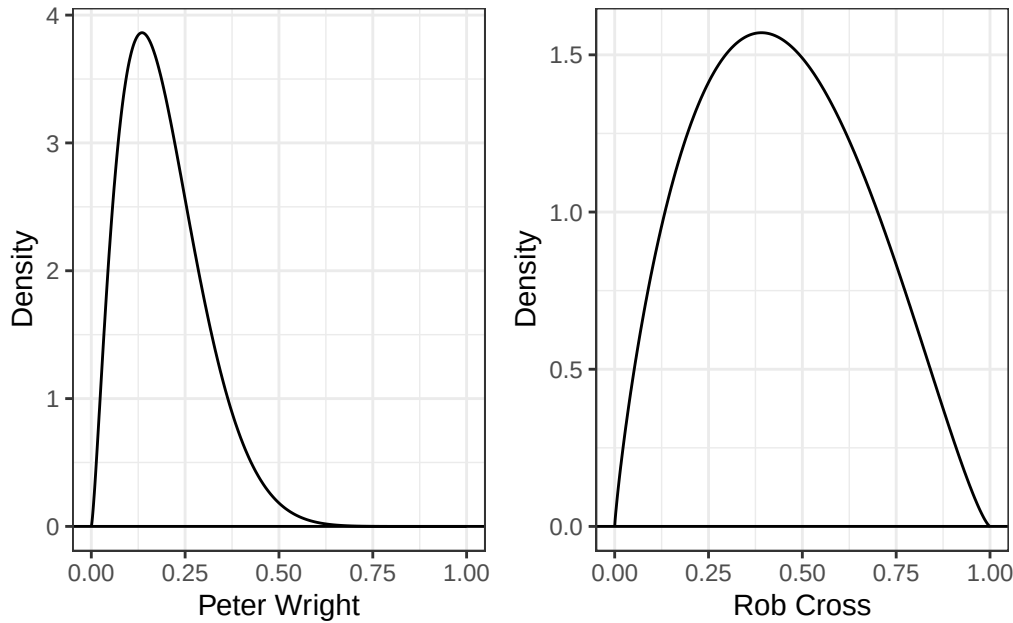
```

1 Beta.plot(4) | Beta.plot(5) | Beta.plot(6)

```



```
1 Beta.plot(7) | Beta.plot(8)
```



The data represent the maximum likelihood of alpha, beta, and theta for each player. Alpha and beta determines the probability of winning each match for the player if their results follows Altham's Multiplicative Beta Binomial Distribution.

5.4 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) data-driven summary of the 2024 Premier League season using your work in Lab 10. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

For each player in the 2024 Premier League season data, I compute the number of matches and the total legs they played, and I calculate the proportion of matches and legs they won. The winning proportion of total matches range

from 0.11 to 0.67. Luke Littler has the highest proportion, and Peter Wright has the lowest proportion. The winning proportion of total legs range from 0.37 to 0.56, and it contains less variation. Most players has a winning proportion around 0.5. Even though Peter Wright still has the lowest proportion, his leg winning proportion(0.37) is a lot higher than his match winning proportion (0.11).

I use the number of legs won in each match to compute the maximum likelihood estimates for p and θ for each player, assuming the probability of the number of legs each player win in a match follows Altham's Multiplicative Binomial Distribution. The result demonstrates that the probability of each player winning a leg is mostly consistent with the result from winning proportion, as Luke Littler has the highest probability (0.59) and Peter Wright has the lowest probability (0.24). The higher the θ is, the more likely for players to win matches with results 6-5 than 6-0. For most players, it is around 1.1, and Peter Wright has the highest θ (1.30), and Rob Cross has the lowest θ (1.04).

I repeat the process but with assuming the probability follows Altham's multiplicative Beta Binomial Distribution. I estimated the parameter α , β , and θ . Additionally, I calculate the mean and variance for the beta distribution for each player. Mean represent the average probability of player winning each leg in each matches, and the result is similar as Luke Littler has the highest mean and Peter Wright has the lowest value. Standard deviation for most player is around 0.2, except for Peter Wright (0.11), representing his probability of winning each leg has less variation.